Studio Nx

JS 28 — initContactForm()

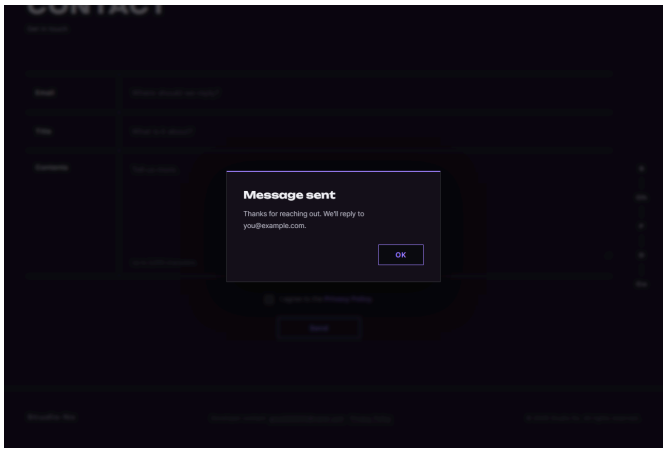
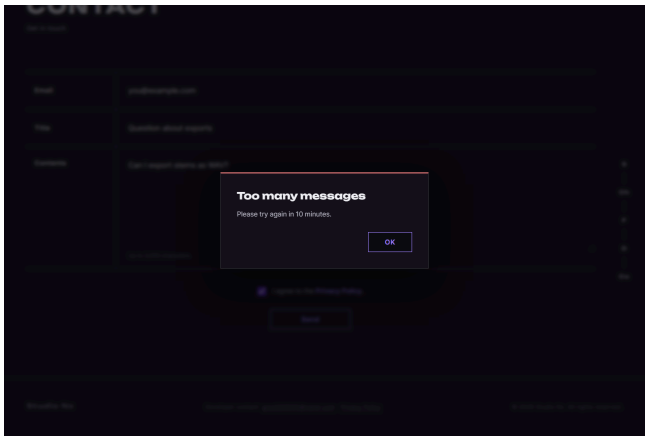
2026-10-08

JS 28 — `initContactForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-12 문의 — 홈 Contact 폼 보내기 · 로그인 시 이메일 자동 채움 · PRD 01 FR-112 · FR-113 (P1-11 화면의 남은 JS)
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	27 `initLogoutButton()` · 29 `formatRelativeTime()` (P1-14)

1. 하는 일

홈 화면 **Contact** 폼의 **Send** 를 서버(백엔드 53)에 연결하고, 결과를 알림창으로 알려 준다.

보냄	이미 3번째 (10분에 2번 넘음)
	

상황	화면
로그인한 사람이 홈을 열면	이메일 칸에 **내 이메일이 미리 채워짐** (비어 있을 때만)
빈 칸 · 형식 · 동의 안 함	보내지 않고 알림창 (FIX-01 19 — 브라우저 검사)
보내는 중	Send 잠금
성공	알림 "Message sent — Thanks for reaching out. We'll reply to you@example.com." · 폼 비움 (내 이메일은 다시 채움)
10분에 3번째 (서버 429)	빨간 알림 "Too many messages — Please try again in 10 minutes." · 폼은 그대로
그 밖의 서버 오류	빨간 알림 "Couldn't send your message" + 서버 문구 · 폼 그대로

2. 언제 쓰나

home.html 이 열릴 때 한 번. 이어지는 것들:

순서	하는 곳
① 칸 검사 · 알림	브라우저 기본 검사 + FIX-01 19 `initFormAlerts` — 통과해야 `submit` 이 일어남

순서	하는 곳
② 보내기	백엔드 53 `POST /inquiries` → 50 · 51 · 52
③ 결과	FIX-01 18 `showAlert`
(자동 채움)	P1-02 `isLoggedIn` → P1-08 `GET /users/me`

결과를 알림창으로 보여 주므로 **HTML 은 바꾸지 않았다** (폼에 결과 문구 자리가 없었다).

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash, isLoggedIn } from '../core/auth.js';
import { initFormAlerts, showAlert } from '../core/dialog.js';
import { api } from '../core/api.js';
```

`isLoggedIn` · `showAlert` · `api` 를 이번에 추가했다.

함수

```
export function initContactForm() {
  const form = document.getElementById('contact-form');
  const email = document.getElementById('contact-email');
  const title = document.getElementById('contact-subject');
  const content = document.getElementById('contact-content');
  const agree = document.getElementById('contact-agree');
  if (!form || !email || !title || !content || !agree) return;
  const submit = form.querySelector('button[type="submit"]');
  let myEmail = '';

  // 로그인 상태면 내 이메일을 미리 채움 (기다리지 않음 — 폼은 바로 쓸 수 있음)
  isLoggedIn().then(async (loggedIn) => {
    if (!loggedIn) return;
    try {
      myEmail = (await api.get('/users/me')).email;
    } catch {
      return; // 못 받아도 문의는 직접 적어 보낼 수 있음
    }
    if (email.value === '') email.value = myEmail;
  });

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    if (submit) submit.disabled = true;
    try {
      const sentTo = email.value.trim();
      // 토큰을 붙여 보냄 → 로그인한 회원이면 서버가 문의를 회원과 연결 (FR-112).
      // 토큰이 없거나 틀려도 이 API 는 401 이 아니라 비회원으로 받음 (백엔드 53)
      await api.post('/inquiries', {
        email: sentTo,
```

```
        title: title.value,
        content: content.value,
        privacy_agreed: agree.checked,
      });
      form.reset();
      if (myEmail) email.value = myEmail;
      showAlert('Message sent', `Thanks for reaching out. We'll reply to ${sentTo}.`); // 닫힐 때까지 기다리지 않음 — 버튼은 바로 다시 켬
    } catch (err) {
      if (err.status === 429) {
        const wait = err.details && err.details.retry_after ? err.details.retry_after : 600;
        const minutes = Math.ceil(wait / 60);
        showAlert(
          'Too many messages',
          `Please try again in ${minutes} minute${minutes === 1 ? '' : 's'}.`,
          { tone: 'error' },
        );
      } else {
        showAlert("Couldn't send your message", err.message, { tone: 'error' });
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const wait = err.details?.retry_after ?? 600;
…
const { email: mine } = await api.get('/users/me');
email.value ||= mine;
```

최신 문법	실제 코드	뜻
`err.details?.retry_after ?? 600`	`err.details && err.details.retry_after ? err.details.retry_after : 600`	있으면 그 값, 없으면 600
`const { email: mine } = …`	`myEmail = (await …).email`	응답에서 칸 꺼내며 이름 바꾸기
`a ||= b`	`if (a === '') a = b`	비어 있을 때만 넣기

문법 풀이

import 문법	뜻
`import { …, isLoggedIn } from '../core/auth.js'`	P1-02 06 — 토큰이 없으면 재발급을 시도해 로그인 여부를 알려 줌
`import { …, showAlert } from '../core/dialog.js'`	FIX-01 18 — 알림창

식	뜻
`isLoggedIn().then(async (loggedIn) => { … })`	**기다리지 않고** 결과가 오면 실행 — 그동안에도 폼은 쓸 수 있음
`catch { … }` (괄호 없음)	오류 내용은 쓰지 않을 때 이렇게 줄여 씀 (`api.js` 도 같은 방식)
`form.reset()`	모든 칸을 처음 값으로 (체크박스도 해제)
`Math.ceil(wait / 60)`	초 → 분, 올림 (예: 599초 → 10분)

식	뜻
`` `minute${minutes === 1 ? '' : 's'}` ``	1이면 minute, 아니면 minutes
`showAlert(…)` (`await` 없음)	알림창이 닫히기를 **기다리지 않음** — 아래 §5

4. 흐름

열 때: 로그인 확인 → 로그인이면 내 이메일을 받아, 칸이 비어 있으면 채움. **보낼 때:** (브라우저 검사 통과) → Send 잠금 → 보냄 → 성공이면 폼 비우고(내 이메일 다시) 알림, 429 면 "몇 분 뒤", 그 밖은 서버 문구 → Send 다시 켬.

5. 왜 이렇게

선택	이유
결과를 알림창으로	폼에 결과 문구 자리가 없고, FIX-01 에서 모든 알림을 같은 모양으로 쓰기로 했다
알림창을 **기다리지 않음**	시험에서 찾은 문제: 처음엔 `await showAlert(…)` 였는데, 그러면 사용자가 OK 를 누를 때까지 `finally`(Send 다시 켜기)가 실행되지 않았다
막히거나 실패하면 폼을 비우지 않음	적은 문의가 사라지면 다시 써야 한다
429 는 "몇 분 뒤" 로	서버가 `retry_after`(초)를 준다 (51) — 사람은 분으로 읽는다
토큰을 붙여 보냄 (`api.post` 기본)	로그인한 회원이면 서버(52)가 문의에 `user_id` 를 연결한다 (FR-112). 이 API 는 토큰이 없거나 틀려도 401 이 아니라 비회원으로 받으므로(53) 문의가 막히지 않는다
자동 채움은 칸이 **비어 있을 때만**	사용자가 먼저 다른 주소를 적었으면 덮어쓰지 않는다
자동 채움을 기다리지 않음	로그인 확인 · 내 정보 요청 동안에도 폼을 바로 쓸 수 있게

처음 만든 것과 바꾼 점 — 회원 연결: 처음에는 `{ auth: false }`(토큰 없이)로 보냈는데, 그러면 로그인한 회원의 문의도 **회원과 연결되지 않았다** (FR-112 빈틈). 이 API 는 토큰 문제로 401 을 내지 않으므로 토큰을 붙여 보내도록 고쳤고, DB 에서 회원 문의는 `user_id` 가 연결되고 비회원 문의는 비어 있음을 확인했다.

6. 시험 (실제 home.html · 실제 서버 · 9개)

반복 전송 제한은 서버 메모리에 세므로, 시험마다 시험 서버를 다시 켜 기록을 비웠다.

시험	확인한 것
비회원	이메일 칸 빔
보내기	보내는 동안 Send 잠금 → "Message sent" · 받는 주소 → 폼 비움 · Send 다시 켬
3번째	빨간 "Too many messages" · "Please try again in 10 minutes." · 폼 그대로
서버 오류 (가짜 500)	"Couldn't send your message" · 서버 문구
로그인한 사람	이메일이 미리 채워짐 → 보낸 뒤 폼을 비워도 내 이메일은 다시 채움 → **DB 에서 그 문의의 `user_id` = 그 회원** (비회원 문의는 비어 있음)

처음 6/7 — §5 의 "알림을 기다려 Send 가 잠긴 채" 를 찾아 고친 뒤 7/7, 로그인 시험 2/2. 회원 연결 빈틈(위)도 고친 뒤 다시 돌려 통과. 홈 화면의 앞 시험(P1-07 13 · 14, FIX-01 18 · 19)도 다시 돌려 모두 통과. 시험이 만든 문

의 · 계정은 지움.